

RedNorte

Informe de Pruebas Unitarias — RedNorte

Resumen General

Total: 94 pruebas — 0 fallos — Cobertura ≥94% en todos los componentes

Componente	Pruebas	% Funciones	% Líneas	Resultado
ms-users	24	96.88%	98.43%	✓
ms-waitlist	21	87.04%	94.67%	
ms-notifications	20	100.00%	99.35%	
frontend (Astro + Preact)	29	90.62%	97.97%	

1. ms-users (24 tests)

Cobertura por archivo

Archivo	% Funciones	% Líneas
controllers/user_controller.ts	100.00%	96.88%
services/user_service.ts	87.50%	96.83%
repositories/user_repository.ts	100.00%	100.00%
db/connection.ts	100.00%	100.00%

Pruebas destacadas

- Registro de usuario exitoso
- Rechazo de registro con datos incompletos
- Login exitoso retorna token JWT
- Login rechazado con credenciales inválidas
- Listar todos los usuarios
- Buscar usuario por ID (existente e inexistente)
- Prevención de correo duplicado

Comando

```
cd backend/ms-users && bun test --coverage
```

2. ms-waitlist (21 tests)

Cobertura por archivo

Archivo	% Funciones	% Líneas
controllers/waitlist_controller.ts	100.00%	94.12%
services/waitlist_service.ts	55.56%	80.60%
repositories/waitlist_repository.ts	100.00%	100.00%
rabbitmq/producer.ts	66.67%	93.33%
models/waitlist.ts	100.00%	100.00%

Pruebas destacadas

- Registro de paciente en lista de espera
- Validación de prioridad (1-4)
- Validación de motivo de consulta (≥ 5 caracteres)
- Actualización de estado (waiting $\rightarrow$ attending $\rightarrow$ finished $\rightarrow$ cancelled)
- Listar pacientes en espera
- Obtener entradas del usuario autenticado (GET /waitlist/mine)
- Validación de JWT en handlers
- Publicación de eventos en RabbitMQ

Comando

```
cd backend/ms-waitlist && bun test --coverage
```

3. ms-notifications (20 tests)

Cobertura por archivo

Archivo	% Funciones	% Líneas
services/notification_service.ts	100.00%	100.00%
repository/notification_repository.ts	100.00%	100.00%
websocket/connectionManager.ts	100.00%	100.00%
rabbitmq/consumer.ts	100.00%	96.77%
db/connection.ts	100.00%	100.00%

Pruebas destacadas

- Consumir mensaje válido de RabbitMQ, persistir y hacer ack
- Rechazar mensajes mal formados (nack sin reencolar)
- Reconexión automática de RabbitMQ
- Gestión de conexiones WebSocket (agregar/remover)
- Envío de notificación en tiempo real vía WebSocket
- Persistencia de notificaciones en BD
- Marcar notificación como leída
- Manejo de usuarios offline

Comando

```
cd backend/ms-notifications && bun test --coverage
```

4. Frontend (29 tests)

Cobertura por archivo

Archivo	% Funciones	% Líneas
components/LoginForm.tsx	100.00%	100.00%
hooks/useNotifications.ts	88.88%	97.56%
lib/api.ts	81.81%	95.23%

Archivo	% Funciones	% Líneas
stores/auth.ts	100.00%	100.00%
stores/notifications.ts	100.00%	100.00%

Pruebas destacadas

- Renderizado del formulario de login
- Manejo de error en login (muestra mensaje)
- Login exitoso redirige a /waitlist
- Estado global de autenticación (login/logout/init)
- Gestión de notificaciones (agregar, marcar leídas, contador)
- Conexión WebSocket con auto-reconnect
- API: login, register, getUsers, getQueue, getMyQueue, addPatient, updateStatus
- ApiError con status code y mensaje

Comando

```
cd frontend && bunx vitest run --coverage
```

Ejecutar Todas las Pruebas

```
# Backend
cd backend/ms-users && bun test --coverage
cd backend/ms-waitlist && bun test --coverage
cd backend/ms-notifications && bun test --coverage

# Frontend
cd frontend && bunx vitest run --coverage
```

Para generar reporte HTML de cobertura en el frontend:

```
cd frontend && bunx vitest run --coverage --reporter=html
# Abrir frontend/coverage/index.html en el navegador
```